

UFRN - Universidade Federal do Rio Grande do Norte

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 17 - PROGRAMAÇÃO EM MEMÓRIA DISTRIBUÍDA

1. Enunciado

Reimplementar a multiplicação matriz-vetor ($y = A \cdot x$) distribuída via MPI, agora particionando a matriz A por colunas em vez de por linhas (versão da Tarefa 16). A distribuição deve usar um tipo derivado definido com `MPI_Type_vector` e `MPI_Type_create_resized` para representar blocos de colunas, distribuídos com `MPI_Scatter`. O vetor x deve ser distribuído em segmentos correspondentes às colunas de cada processo. Cada processo calcula uma contribuição parcial para todos os elementos do vetor y, e essas contribuições são somadas no processo 0 com `MPI_Reduce` e a operação `MPI_SUM`. O trabalho pede ainda uma discussão sobre as diferenças de acesso à memória e de desempenho entre a distribuição por colunas e a distribuição por linhas.

2. Fundamentação Teórica

2.1 Distribuição por linhas (Tarefa 16 — referência)

Na versão por linhas, a matriz A ($M \times N$, armazenada em row-major) é dividida em blocos contíguos de linhas entre os P processos com `MPI_Scatter` simples (linhas já são contíguas na memória, dispensando tipo derivado). O vetor x é replicado em todos os processos via `MPI_Bcast`, pois cada linha depende do vetor x completo. Cada processo calcula diretamente o subvetor de y que lhe cabe, com um acumulador escalar por linha, e os resultados são reunidos no processo 0 com `MPI_Gather`.

2.2 Distribuição por colunas

Na versão por colunas, cada processo recebe um bloco de colunas inteiras de A. Como a matriz é armazenada em row-major, uma coluna não é contígua na memória — seus elementos estão espaçados de N posições (o número de colunas). Para descrever esse padrão, foi definido o tipo derivado:

```
MPI_Type_vector((int)M, 1, (int)N, MPI_DOUBLE, &coluna_tmp);
```

Esse tipo descreve M blocos de 1 elemento, cada um a N posições de distância do anterior exatamente o padrão de uma coluna. Entretanto, o extent natural desse tipo cobre do primeiro ao

último elemento da coluna quase a matriz inteira, o que impediria o MPI_Scatter de posicionar corretamente colunas adjacentes para processos diferentes. Por isso o extent é redefinido para o tamanho de um único double:

```
MPI_Type_create_resized(coluna_tmp, 0, sizeof(double), &tipo_coluna);
```

Com o extent redimensionado, count = local_N desse tipo corresponde exatamente a local_N colunas consecutivas (0, 1, 2, ...), cada uma iniciando logo após o começo da anterior o que permite usar MPI_Scatter normalmente, um bloco de colunas por processo.

2.3 Ordem das operações e diferenças de acesso à memória

As duas versões diferem tanto no padrão de acesso à memória quanto no padrão de comunicação:

- Acesso a y: na versão por linhas, cada $y[i]$ é acumulado uma única vez em registrador ao longo do laço sobre j (padrão dot-product), e escrito uma única vez na memória. Na versão por colunas, cada processo percorre suas colunas locais e, para cada uma, lê e reescreve o vetor y_{parcial} inteiro (padrão saxpy: $y_{\text{parcial}}[i] += A[i][j]*x[j]$ para todo i). Isso significa que y_{parcial} (tamanho M) é revisitado local_N vezes, gerando muito mais tráfego de memória quando M é grande o suficiente para não caber em cache.
- Comunicação de entrada (x): na versão por linhas, x precisa ser replicado inteiro em todos os processos (MPI_Bcast), pois cada linha usa todos os elementos de x. Na versão por colunas, cada processo só precisa do segmento de x referente às suas colunas MPI_Scatter simples, sem replicação.
- Comunicação de saída (y): na versão por linhas, cada processo já produz o pedaço final de y que lhe cabe, bastando um MPI_Gather (movimentação pura de dados). Na versão por colunas, cada processo produz uma contribuição parcial para y inteiro, exigindo MPI_Reduce com MPI_SUM uma operação de redução sobre M elementos, mais cara que um Gather equivalente, e cujo custo não diminui proporcionalmente a P.
- Overhead do tipo derivado: a versão por linhas não precisa de tipo derivado. A versão por colunas exige empacotamento/packing de elementos não contíguos dentro do MPI_Scatter, um custo adicional que a versão por linhas não paga.

3. Experimento

Ambiente: partição amd-512 do NPAD, nó único, execução exclusiva (--exclusive), até 32 tasks MPI, tempo limite de 30 minutos. Compilação com mpicc -O3.

Job da Tarefa 17 (distribuição por colunas): job 2129973. Tamanhos de matriz testados: 512, 1024, 2048 e 4096 ($M=N$), com $P \in \{1, 2, 4, 8, 16, 32\}$ pulando combinações em que M não é divisível por P

. Cada combinação foi repetida 30 vezes, reportando o tempo médio.

Job da Tarefa 16 (distribuição por linhas, referência): job 2125273, mesmo ambiente e mesma metodologia, incluindo também $M=N=8192$, com o mesmo esquema de repetições.

O script SLURM utilizado para a Tarefa 17 está no Anexo B, e o código-fonte completo (multMxV.c) no Anexo A.

4. Resultados

4.1 Tarefa 17 — distribuição por colunas

P	M=N	reps	tempo médio (s)
1	512	30	0.000813139
2	512	30	0.000854855
4	512	30	0.001623386
8	512	30	0.002301628
16	512	30	0.002418880
32	512	30	0.002496479
1	1024	30	0.003653634
2	1024	30	0.003801897
4	1024	30	0.006737998
8	1024	30	0.010022640
16	1024	30	0.009537418
32	1024	30	0.009687272
1	2048	30	0.021020993
2	2048	30	0.019222461
4	2048	30	0.027830549
8	2048	30	0.042063919
16	2048	30	0.041452590
32	2048	30	0.043066226
1	4096	30	0.152239967
2	4096	30	0.156544268
4	4096	30	0.152365597
8	4096	30	0.198966109
16	4096	30	0.193968005
32	4096	30	0.210759653

4.2 Tarefa 16 — distribuição por linhas (referência)

P	M=N	reps	tempo médio (s)
1	512	30	0.000348034
2	512	30	0.000301912
4	512	30	0.000196084
8	512	30	0.000144912
16	512	30	0.000140234
32	512	30	0.000183668
1	1024	30	0.001356880
2	1024	30	0.000932088

P	M=N	reps	tempo médio (s)
4	1024	30	0.000791086
8	1024	30	0.000802781
16	1024	30	0.000343208
32	1024	30	0.000298183
1	2048	30	0.006018828
2	2048	30	0.004066439
4	2048	30	0.002708305
8	2048	30	0.002654833
16	2048	30	0.000906723
32	2048	30	0.000818427
1	4096	30	0.028260827
2	4096	30	0.018992681
4	4096	30	0.012481476
8	4096	30	0.010659952
16	4096	30	0.002943924
32	4096	30	0.002190432
1	8192	30	0.113708585
2	8192	30	0.078263721
4	8192	30	0.059852758
8	8192	30	0.054659096
16	8192	30	0.022300169
32	8192	30	0.018214053

4.3 Gráficos

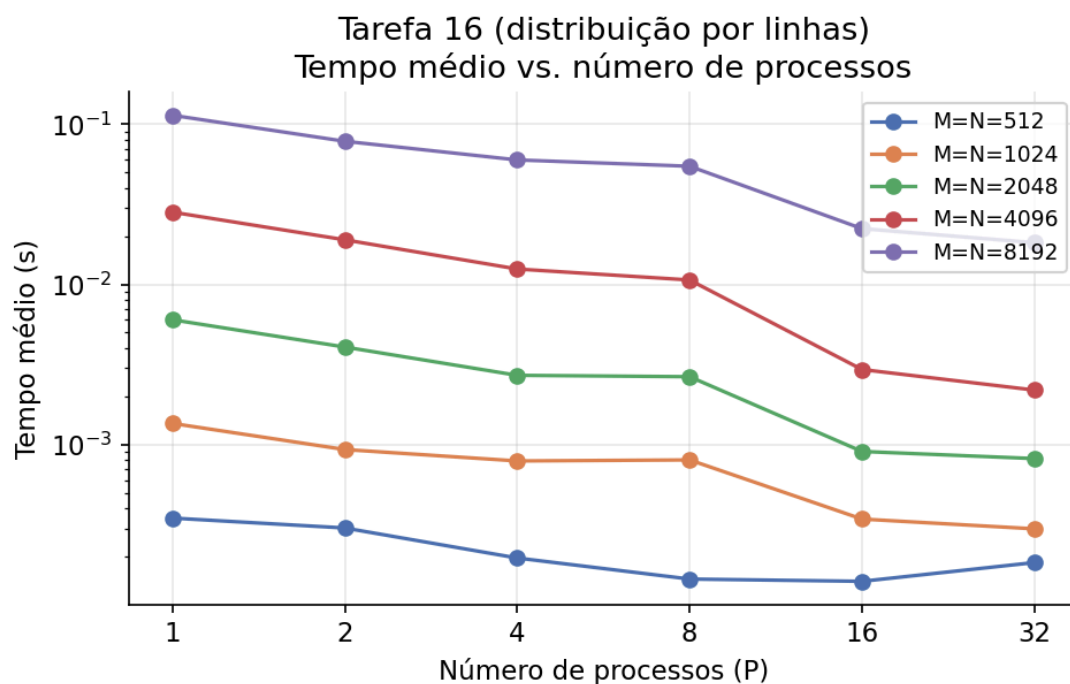


Figura 1 — Tarefa 16 (linhas): tempo médio vs. número de processos, para cada tamanho de matriz (escala log-log).

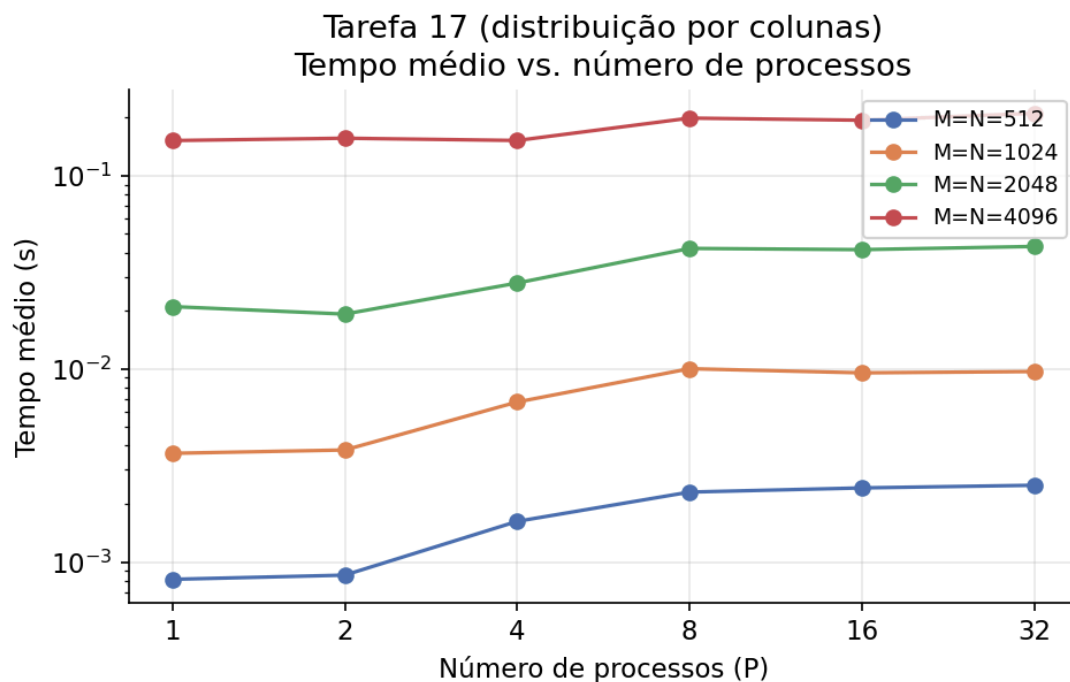


Figura 2 — Tarefa 17 (colunas): tempo médio vs. número de processos, para cada tamanho de matriz (escala log-log).

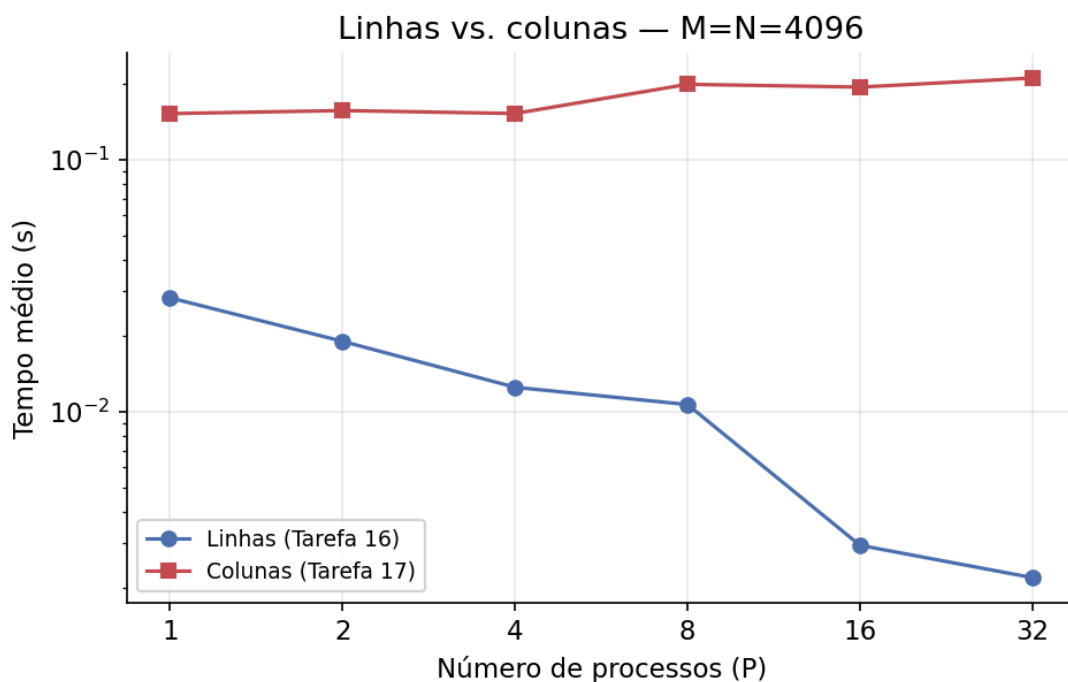


Figura 3 — Comparação direta entre linhas e colunas para $M=N=4096$.

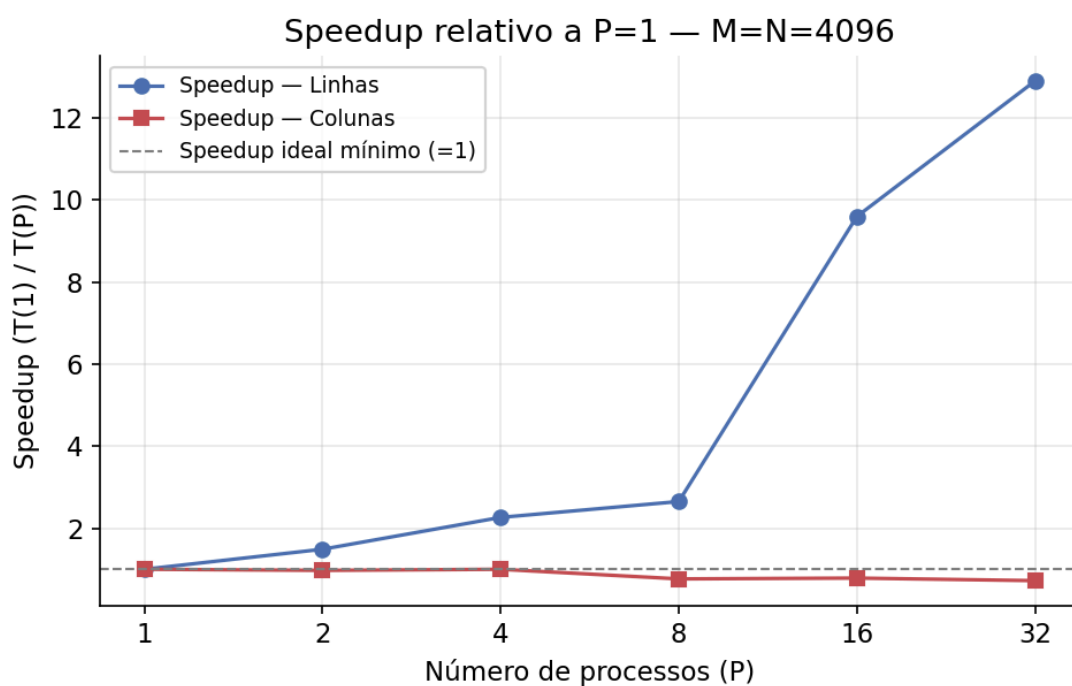


Figura 4 — Speedup relativo a $P=1$ para $M=N=4096$. A linha tracejada marca o speedup mínimo ($=1$); valores abaixo dela indicam desaceleração.

5. Discussão

Os resultados confirmam, de forma bastante acentuada, a assimetria prevista na Seção 2.3 entre as duas estratégias de distribuição e escalabilidade oposta na Tarefa 16, o tempo cai de forma a não haver variação com o aumento de P para todos os tamanhos de matriz testados, com ganhos

expressivos a partir de $P=16$ (speedup em $M=N=4096$, $P=32$). Já na Tarefa 17, o tempo permanece praticamente constante ou piora com mais processos o speedup cai abaixo de 1 já em $P=8$ para $M=N=4096$, e o mesmo padrão se repete nas demais matrizes (Figuras 2 e 4).

A diferença já aparece em $P=1$ não é apenas overhead de comunicação: mesmo sem nenhum paralelismo, a versão por colunas é de 2,3x ($M=512$) a 5,4x ($M=4096$) mais lenta que a versão por linhas rodando serialmente. Isso é consistente com a análise da Seção 2.3 o padrão da versão por colunas é uma multiplicação de uma matriz por um vetor y_{parcial} inteiro revisitado a cada coluna, enquanto a versão por linhas é produto por escalar um acumulador em registrador, $y[i]$ escrito uma única vez. Como a matriz é armazenada em row-major, o acesso por colunas também depende do tipo derivado para reunir elementos espalhados na memória, algo que a versão por linhas dispensa.

O hiato cresce com o tamanho da matriz: a razão $\text{tempo}(\text{colunas})/\text{tempo}(\text{linhas})$ sobe de aproximadamente 2,3x em $M=512$ para aproximadamente 96x em $M=4096$ ($P=32$). Isso é compatível com a hipótese de que, à medida que M cresce, o acesso repetido a y_{parcial} aumenta o tráfego na hierarquia de memória e pode aumentar a pressão sobre os caches, especialmente para matrizes maiores., cada uma das local_N colunas processadas revisita esse vetor inteiro na memória, multiplicando o tráfego de memória proporcionalmente a $\text{local_N} \times M$, muito mais sensível ao crescimento de M do que o acesso por linhas pois esse tipo de acesso quebra a localidade espacial.

Com mais processos, o custo fixo de comunicação passa a dominar na versão por colunas, o `MPI_Scatter` precisa empacotar dados não contíguos (via o tipo derivado da coluna) e o `MPI_Reduce` final soma vetores de tamanho M inteiro a cada repetição dois custos que não diminuem proporcionalmente a P o Reduce, em particular, sempre opera sobre M elementos, não M/P . Como o tempo de computação por processo cai com P menos colunas cada, mas o custo de comunicação/redução não cai na mesma proporção, a partir de um certo P a comunicação passa a dominar o tempo total o que explica a estagnação e a leve piora observada em $P=16$ e $P=32$.

6. Conclusão

A distribuição por colunas, apesar de ser um exercício didático valioso para praticar tipos derivados (`MPI_Type_vector`, `MPI_Type_create_resized`) e operações de redução (`MPI_Reduce/MPI_SUM`), mostrou-se, para esta matriz armazenada em row-major, sistematicamente pior que a distribuição por linhas tanto em desempenho serial quanto em escalabilidade. A causa não está apenas na comunicação MPI, mas principalmente no padrão de acesso à memória: a versão por colunas revisita o vetor de saída inteiro a cada coluna processada, enquanto a versão por linhas escreve cada elemento de saída uma única vez. Somado a isso, o custo de empacotamento do tipo derivado no Scatter e o custo do Reduce sobre o vetor completo não diminuem proporcionalmente ao número de processos, fazendo a comunicação dominar o tempo total a partir de P moderado. Na prática, para matrizes row-major como a deste experimento, a distribuição por linhas deve ser preferida sempre que o objetivo for desempenho a distribuição por colunas só se torna vantajosa em cenários onde a matriz é naturalmente armazenada em column-major ou onde o acesso column-wise é inerente ao problema.

Anexo A — Código-fonte (multMxV.c)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

static double *aloca_vetor(long n) {
    double *v = (double *) malloc((size_t) n * sizeof(double));
    if (!v) {
        fprintf(stderr, "Falha ao alocar %ld doubles\n", n);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    return v;
}

/* Gera valores pseudo-aleatorios determinísticos (mesma semente sempre),
 * para que as execucoes sejam reprodutíveis e comparáveis entre si. */
static void preenche_aleatorio(double *v, long n, unsigned int seed) {
    unsigned int state = seed;
    for (long i = 0; i < n; i++) {
        state = state * 1103515245u + 12345u;
        v[i] = (double) ((state >> 8) % 1000) / 100.0; /* [0, 10) */
    }
}

static void multiplica_serial(const double *A, const double *x, double *y,
                             long M, long N) {
    for (long i = 0; i < M; i++) {
        double soma = 0.0;
        const double *linha = A + i * N;
        for (long j = 0; j < N; j++) {
            soma += linha[j] * x[j];
        }
        y[i] = soma;
    }
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, nprocs;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    if (argc < 3) {
        if (rank == 0) {
            fprintf(stderr,
                    "Uso: %s M N [reps] [--verify]\n", argv[0]);
        }
        MPI_Finalize();
        return 1;
    }

    long M = atol(argv[1]);
    long N = atol(argv[2]);
    int reps = (argc >= 4 && argv[3][0] != '-') ? atoi(argv[3]) : 10;
    int verify = 0;
    for (int i = 3; i < argc; i++) {
        if (strcmp(argv[i], "--verify") == 0) verify = 1;
    }

    /* Agora quem precisa ser divisível pelo numero de processos e o
     * numero de COLUNAS (N), pois é ela que será particionada entre
     * os processos com MPI_Scatter + tipo derivado. */
    if (N % nprocs != 0) {
        if (rank == 0) {
```

```

        fprintf(stderr, "Erro: N=%ld precisa ser divisivel pelo numero de " "processos
(nprocs=%d) para usar MPI_Scatter com blocos de " "colunas de tamanho igual.\n", N, nprocs);
    }
    MPI_Finalize();
    return 1;
}

long local_N = N / nprocs;

/* ----- Tipo derivado: uma coluna de A ----- */
/* A e M x N, armazenada em row-major (A[i*N+j]). Uma coluna j tem
 * M elementos, cada um a N posicoes (doubles) de distancia do
 * anterior: MPI_Type_vector(M blocos, 1 elemento por bloco, stride N). */
MPI_Datatype coluna_tmp, tipo_coluna;
MPI_Type_vector((int) M, 1, (int) N, MPI_DOUBLE, &coluna_tmp);

/* O extent natural desse tipo vai do 1o ao ultimo elemento da coluna
 * (quase a matriz inteira), o que faria o MPI_Scatter posicionar
 * cada "unidade" de coluna muito distante da seguinte. Redefinindo
 * o extent para 1 double, count=local_N desse tipo redimensionado
 * passa a corresponder a local_N colunas ADJACENTES (col 0,1,2,...),
 * cada uma comecando logo apos o inicio da anterior. */
MPI_Type_create_resized(coluna_tmp, 0, sizeof(double), &tipo_coluna);
MPI_Type_commit(&tipo_coluna);
MPI_Type_free(&coluna_tmp);

/* Apenas o processo 0 possui a matriz completa e o vetor y completo */
double *A = NULL, *x = NULL, *y = NULL;
double *local_x = aloca_vetor(local_N);
/* local_A guarda local_N colunas de M elementos cada, entregues pelo
 * MPI_Scatter de forma contigua por coluna (column-major local):
 * local_A[jl*M + i] = A[i*N + (rank*local_N + jl)]. */
double *local_A = aloca_vetor(local_N * M);
double *local_y_parcial = aloca_vetor(M);

if (rank == 0) {
    A = aloca_vetor(M * N);
    x = aloca_vetor(N);
    y = aloca_vetor(M);
    preenche_aleatorio(A, M * N, 42u);
    preenche_aleatorio(x, N, 7u);
}

double tempo_total = 0.0;

for (int r = 0; r < reps; r++) {
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    /* Distribui blocos de local_N colunas por processo */
    MPI_Scatter(A, (int) local_N, tipo_coluna,
                local_A, (int) (local_N * M), MPI_DOUBLE,
                0, MPI_COMM_WORLD);

    /* Distribui os segmentos de x correspondentes as colunas de cada
     * processo (contiguo em x, dispensa tipo derivado). */
    MPI_Scatter(x, (int) local_N, MPI_DOUBLE,
                local_x, (int) local_N, MPI_DOUBLE,
                0, MPI_COMM_WORLD);

    /* Cada processo contribui para TODOS os M elementos de y:
     * y_parcial[i] += soma_{j local} A[i][j] * x[j] */
    memset(local_y_parcial, 0, (size_t) M * sizeof(double));
    for (long jl = 0; jl < local_N; jl++) {
        double xj = local_x[jl];
        const double *coluna = local_A + jl * M;
        for (long i = 0; i < M; i++) {
            local_y_parcial[i] += coluna[i] * xj;
        }
    }
}

```

```

/* Soma elemento a elemento as contribuicoes parciais de todos os
 * processos, entregando o y completo (tamanho M) no processo 0. */
MPI_Reduce(local_y_parcial, y, (int) M, MPI_DOUBLE, MPI_SUM,
           0, MPI_COMM_WORLD);

MPI_Barrier(MPI_COMM_WORLD);
double t1 = MPI_Wtime();
tempo_total += (t1 - t0);
}

if (rank == 0) {
    double tempo_medio = tempo_total / reps;
    /* Saida em CSV para facilitar coleta de dados nos benchmarks:
     * nprocs,M,N,reps,tempo_medio_s */
    printf("%d,%ld,%ld,%d,%.9f\n", nprocs, M, N, reps, tempo_medio);

    if (verify) {
        double *y_serial = aloca_vetor(M);
        multiplica_serial(A, x, y_serial, M, N);
        double erro_max = 0.0;
        for (long i = 0; i < M; i++) {
            double dif = fabs(y[i] - y_serial[i]);
            if (dif > erro_max) erro_max = dif;
        }
        fprintf(stderr, "[verify] erro absoluto maximo entre paralelo e serial: %e\n",
erro_max);
        free(y_serial);
    }
}

MPI_Type_free(&tipo_coluna);

free(local_x);
free(local_A);
free(local_y_parcial);
if (rank == 0) {
    free(A);
    free(x);
    free(y);
}

MPI_Finalize();
return 0;
}

```

Anexo B — Script SLURM (script.sh)

```
#!/bin/bash
#SBATCH --job-name=tarefa17
#SBATCH --partition=amd-512
#SBATCH --nodes=1
#SBATCH --exclusive
#SBATCH --ntasks=32
#SBATCH --time=00:30:00
#SBATCH --output=tarefa17_%j.out
#SBATCH --error=tarefa17_%j.err

set -e

mpicc -O3 -o multMxV multMxV.c -lm

REPS=30
OUT="resultados_tarefa17_${SLURM_JOB_ID}.csv"
echo "nprocs,M,N,reps,tempo_medio_s" > "$OUT"

TAMANHOS=(512 1024 2048 4096)
NPROCS_LIST=(1 2 4 8 16 32)

for M in "${TAMANHOS[@]}; do
    N=$M
    for P in "${NPROCS_LIST[@]}; do
        if [ $((M % P)) -ne 0 ]; then
            continue
        fi
        mpirun -np "$P" ./multMxV "$M" "$N" "$REPS" >> "$OUT"
    done
done

echo "Concluido. Resultados em $OUT"
```